



Multi-Cloud Disaster Recovery with Pulumi



Arpita Chowdhury

```
Saved config
Installing dependencies...

added 355 packages, and audited 356 packages in 12s
36 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
Finished installing dependencies

Your new project is ready to go! 🎉
To perform an initial deployment, run `pulumi up`
arpitachowdhury@Arpitas-MacBook-Air infrastructure %
```



Introducing Today's Project!

In this project, I'm going to use Pulumi with TypeScript to manage both AWS and GCP infrastructure from a single codebase and build a multi-cloud disaster recovery system with automatic failover using CloudFront. Multi-cloud disaster recovery is important because even a multi-region setup can fail if a single cloud provider has an outage, so running workloads across different clouds ensures higher availability and resilience. Infrastructure as Code helps by making all infrastructure changes version-controlled, repeatable, and consistent across environments, preventing configuration drift and enabling fast recovery and rollback during failures.

Key tools and concepts

In this step, I'm going to install the Pulumi CLI and initialize a new TypeScript project to manage my cloud infrastructure as code. I will import my existing App Runner services in us-east-1 and us-west-2 into Pulumi so that the current deployed resources are represented in code instead of being managed manually in the console. Importing infrastructure means bringing already-created cloud resources under Pulumi's management without recreating them. This is useful because it prevents configuration drift, makes infrastructure version-controlled and reproducible, and allows me to manage multi-region and multi-cloud deployments consistently from a single codebase.

Challenges and wins

In this step, I'm going to deploy my app to GCP Cloud Run after configuring GCP credentials and enabling required APIs. Multi-cloud is important because it provides true disaster recovery and avoids relying on a single cloud provider.



Arpita Chowdhury

NextWork Student

nextwork.org

I'll import the Cloud Run service into Pulumi so both AWS and GCP infrastructure are managed as code in one place, preventing configuration drift. Finally, I'll update the app to detect when it's running on GCP so I can verify multi-cloud failover and routing behavior.



Initializing a Pulumi TypeScript Project

I created a Pulumi project by running `pulumi new aws-typescript` in my infrastructure directory, which generated a TypeScript-based Infrastructure as Code project. The stack name I chose was `dev`, which represents my development environment configuration. Pulumi stores state in the Pulumi Cloud backend (app.pulumi.com), which tracks the deployed resources, configuration values, and history of changes so infrastructure can be versioned, audited, and reproduced consistently across environments.

```
arpitachowdhury@Arpita-MacBook-Air infrastructure % pulumi preview
Previewing update (dev)

View in Browser (Ctrl+O): https://app.pulumi.com/Chyarpsh-org/multi-region-app-infra/dev/previews/9fc40e17-ae9b-41d2-a10d-9e1fae7d837f

Type                                Name                                Pl
+-----+-----+-----+
+ pulumi:pulumi:Stack                multi-region-app-infra-dev         cr
+   | pulumi:providers:aws            east                               cr
+   | pulumi:providers:aws            west                               cr
+   | aws:apprunner:Service            primary-apprunner                  cr
+   | aws:cloudfront:Distribution      main-cdn                           cr
+   | aws:apprunner:Service            secondary-apprunner                 cr

Resources:
+ 6 to create

arpitachowdhury@Arpita-MacBook-Air infrastructure % 4
```

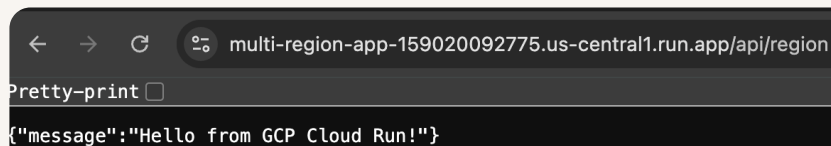
Importing existing AWS infrastructure

I imported my App Runner services by running `pulumi import` for each App Runner service ARN (`us-east-1` and `us-west-2`). Then I ran `pulumi preview`, which showed a deployment plan for my stack. This confirms Pulumi is successfully reading my TypeScript IaC code and can generate an infrastructure plan from it (next I need to verify the resources are tracked as imports, not new creates).



Deploying to GCP Cloud Run

I deployed the application to Google Cloud Run using the `gcloud run deploy` command with source-based deployment, and when I accessed the `/api/region` endpoint it returned “Hello from GCP Cloud Run!”, which confirms that the service is running successfully on GCP Cloud Run and the application correctly detects the GCP environment.



A screenshot of a web browser window. The address bar shows the URL `multi-region-app-159020092775.us-central1.run.app/api/region`. Below the address bar, there is a 'Pretty-print' checkbox. The main content area of the browser displays a JSON response: `{"message": "Hello from GCP Cloud Run!"}`.

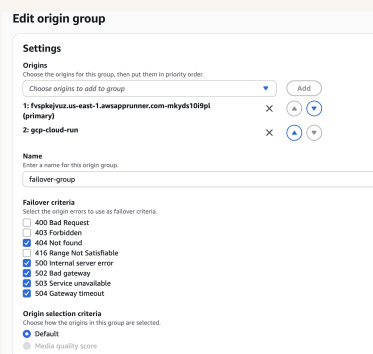
Managing multi-cloud infrastructure with Pulumi

I imported the existing Cloud Run service into Pulumi using `pulumi import` (after deploying it with `gcloud run deploy`), so Pulumi can “adopt” the live GCP resource instead of creating a new one. Now my Pulumi program manages both AWS + GCP infrastructure from one TypeScript codebase—including my AWS App Runner services/CloudFront setup and my GCP Cloud Run service—so any future changes are tracked, repeatable, and deployable through `pulumi preview/up` (instead of manual console clicks). This is multi-cloud IaC because the same version-controlled Pulumi project defines and manages infrastructure across two cloud providers (AWS + GCP), helping prevent configuration drift and making failover/DR reproducible.



Configuring CloudFront Multi-Cloud Failover

In this step, I'm going to configure CloudFront Origin Groups so the primary origin is AWS App Runner (us-east-1) and the failover origin is GCP Cloud Run, replacing the old us-west-2 origin. Failover works by having CloudFront continuously route traffic to the primary origin, and automatically switch to the failover origin when the primary starts returning the configured failover errors (like 5xx/4xx you selected) for requests. This gives me disaster recovery because if AWS App Runner becomes unhealthy or unavailable, CloudFront can still serve the same application from GCP Cloud Run, keeping the site online. After updating CloudFront, I'll import the updated CloudFront configuration into Pulumi so the new AWS to GCP setup is tracked as IaC, and then I'll test failover by pausing the AWS App Runner service and confirming traffic switches to GCP.



Setting up origin groups for automatic failover

I configured the origin group with two origins in priority order and a custom failover policy. The primary origin is the AWS App Runner service in us-east-1. The secondary (failover) origin is GCP Cloud Run.



Arpita Chowdhury

NextWork Student

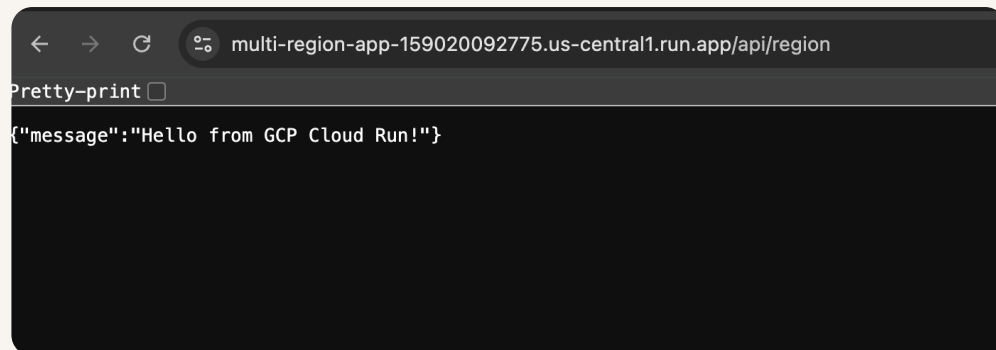
nextwork.org

Failover triggers when CloudFront receives 404, 500, 502, 503, or 504 errors from the primary origin, at which point traffic is automatically routed to GCP Cloud Run.



Testing Multi-Cloud Disaster Recovery

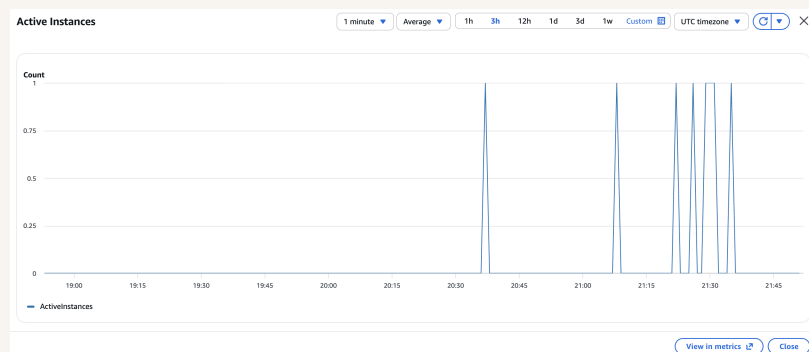
I tested failover by pausing my AWS App Runner service (primary origin) and then reloading the CloudFront /api/region endpoint. When AWS was paused, CloudFront detected the primary origin was unhealthy and automatically failed over to the secondary origin (GCP Cloud Run), returning the “Hello from GCP Cloud Run!” response. This proves my DR setup works because traffic continues to be served even when the primary cloud service is unavailable, demonstrating high availability and automatic multi-cloud failover through CloudFront origin groups.





Building a CloudWatch Monitoring Dashboard

In this secret mission, I'm going to use Cursor to generate Pulumi code that defines an AWS CloudWatch dashboard for my multi-cloud architecture and deploy it using `pulumi up`. The dashboard will show real-time metrics such as request counts, error rates, and origin health for CloudFront and the backend services across AWS and GCP. This helps with DR because it provides visibility into system behavior during normal operation and failover events, allowing me to confirm that traffic shifts correctly when a primary service becomes unavailable.



Real-time observability for disaster recovery

I deployed the dashboard using Pulumi by running `pulumi up`, which provisioned a CloudWatch dashboard to visualize real-time metrics for my multi-cloud architecture. It shows the number of active backend instances over time, allowing me to observe how capacity changes as traffic is served by different origins. When I tested failover, the metric briefly dropped and then showed spikes in active instances, indicating that the primary AWS service became unavailable and CloudFront routed traffic to the secondary GCP Cloud Run service, which scaled up to handle requests.



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

